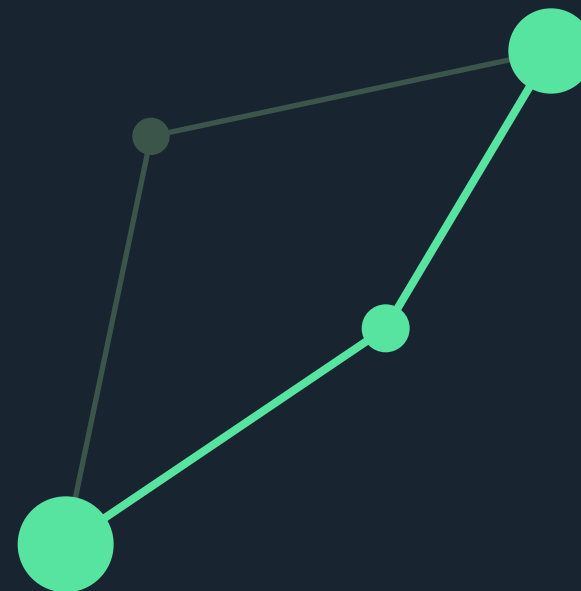


ShareOJ

考える楽しさを、
次の一問へ。

プロダクト概要と技術構成

ENGINEERING INTRODUCTION / 2026.09.22



解く経験を、次の問題と知識へ

01

解く

コードを提出し、
結果から解法を見直す。

02

作る

問題文とテストを用意し、
自作の問題を公開する。

03

共有する

解き方や学んだことを
記事として残す。

04

集まる

コンテストを開催し、
同じ問題に挑戦する。

解答者から作問者へ。ひとつのサービスで活動を広げられる。

問題を読んで、提出し、結果を確かめる

問題を読む



サンプル検証



提出する



結果を確認

CODE / Python の例

```
a, b = map(int, input().split())  
print(a + b)
```

標準入力 → プログラム → 標準出力

RESULT / 表示内容の例

AC

正解

WA

出力が条件を満たさない

TLE

実行時間の制限を超過

作問を支える編集と検証

書く

Markdownと数式で
問題文と解説を編集

確かめる

テストケース、生成コード、入力検証を用意
未公開のまま提出し、テスターと確認

公開する

問題を公開、またはコンテストへ登録
提出時の問題と採点条件を記録

コンテストと記事で、学びを共有する

CONTEST

同じ問題に挑む場

開催期間、問題、配点を設定

得点とペナルティで順位を表示

終了後は問題と登録済み解説を公開

ARTICLE

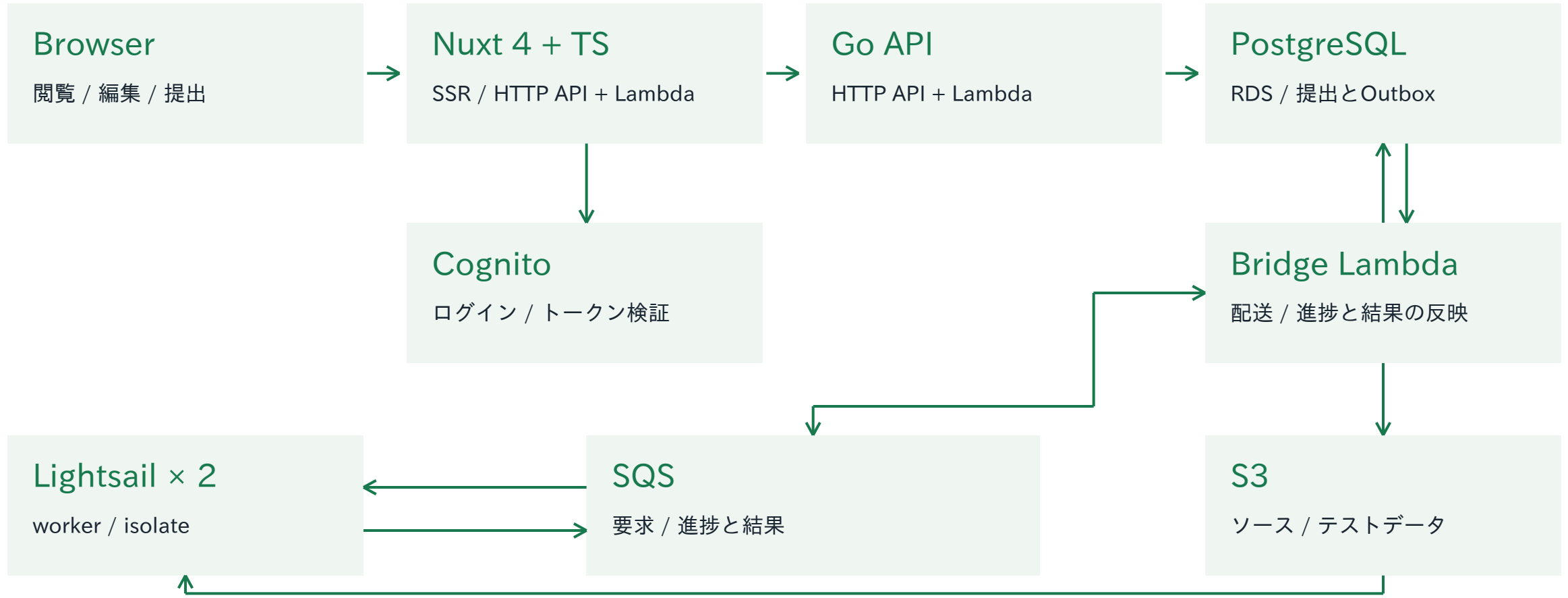
考え方を残す場

解き方や学んだことを記事にする

問題に取り組む人が読める

解答の先にある理解を共有する

Webと採点基盤を分けた全体構成



1件の提出が結果になるまで

01



02



03



04

受付

配送

採点

確定

問題の版、言語、
制限値をDBに保存

S3に入力を置き、
SQSへ要求を送る

入力を検証し、
隔離環境で実行

結果をDBへ反映し、
画面から取得

受付と採点を非同期に分離し、処理中の進捗も返す。

再配信があっても、確定結果を守る

起こり得ること

実装での扱い

API受付後の配送待ち

提出行をOutboxとしてDBに残す

要求や結果の重複

試行IDを照合し、確定結果を上書きしない

ワーカーの途中停止

SQSが再配信し、未完了のまま残さない

「一度だけ実行」ではなく、重複を受け止める設計。

提出コードをケースごとに隔離する

Lightsail host

管理ワーカー

isolate / ケース専用cgroup

実行ファイル + 入力

CPU、メモリ、出力を制限

外部ネットワークへの経路なし

提出側へ渡さない

AWS認証情報

管理ワーカーのコード

期待出力と計測メタデータ

ケース終了後に

作業領域と子孫を回収

境界：提出と管理処理は同じカーネルを共有する。

資源を制限し、判定の根拠を記録する

100–5,000

ms

ケースのCPU時間

64–512

MiB

問題ごとのメモリ上限

1

提出 / ホスト

ホスト内の同時採点

TLE CPU時間と経過時間を別々に監視する。

MLE ケースのcgroup OOMを根拠にする。
ホスト障害や計測不能はJEとして区別する。

問題に合わせて選ぶ3つの判定方法

通常判定

出力を期待出力と比較

入力 → 提出 → 出力比較

スペシャルジャッジ

複数の正解を検証コードで判定

提出 → 出力 → 検証コード

対話形式

質問と応答を交わす問題

提出 ↔ 中継 ↔ 対話用ジャッジ

言語環境を固定し、検証して公開する

言語ガイドに掲載される環境

C / C++ / Rust

Python / Java / C#

Go / Nim / Haskell

JavaScript / TypeScript / Ruby

固定

コンパイラとライブラリの版

検証

実機smokeと資源制限

公開

digestと受付対象を同期

実際に選べる言語は、提出欄と GET /runtimes で確認。

SSRと認証をサーバー側でつなぐ



読み始められる

初回HTMLに問題文と数式を含める

権限を確かめる

認証に加えて、所有者と公開範囲を検証

入力を制御する

Markdownと数式の安全な描画、Origin検証

配布の完了を、提出の成功まで確認する

WEB / API

型検査 → ビルド → ブラウザー検証 → 実DB連携テスト

JUDGE



完了条件 : state.json の status: passed / step: complete

設計上の選択と、現在の制約

採用した構成

運用で考慮すること

Lightsail + isolate

共有カーネル。CPU負荷で計測値が変動し得る。

2台で要求キューを共有

最大2提出を同時採点。更新時は受付を停止。

RDS PostgreSQL

設定はSingle-AZ。自動フェイルオーバーなし。

runtime digestで条件を識別

過去の実行環境を保存する仕組みは未実装。

考える楽しさを、次の一問へ。

解く。作る。共有する。

問題と記事、コンテストをつなぐオンラインジャッジ。

Nuxt + Go + PostgreSQL

S3 / SQS + Lightsail / isolate

www.share-oj.net

参照資料と、このスライドの前提

概要と利用体験

[README.md](#) / [web/app/pages/](#)

WebとAPI

[web/README.md](#) / [api/README.md](#) / [infra/README.md](#)

配送と結果反映

[api/cmd/judge-bridge/](#) / [api/internal/submissions/](#)

採点と隔離

[judge/README.md](#) / [docs/judge/runtime-policy.md](#)

公開と検証

[docs/judge/deployment-checks.md](#) / [.github/workflows/ci.yml](#)

基準 : 2026-09-22 / repository c59a2e1

本番環境の再検証、利用実績や性能の測定は本資料の対象外。